A Computing-in-Memory Engine Supporting One-Shot Floating-Point NN Inference and On-Device Fine-Tuning for Edge AI

Haikang Diao, *Graduate Student Member, IEEE*, Haoyang Luo, *Graduate Student Member, IEEE*, Jiahao Song, *Member, IEEE*, Bocheng Xu, *Graduate Student Member, IEEE*, Runsheng Wang, *Member, IEEE*, Yuan Wang, *Member, IEEE*, and Xiyuan Tang, *Senior Member, IEEE*

***Abstract*— With the rapid advancement of edge AI, the complexity of tasks on edge devices is continually increasing, demanding better efficiency and precision from AI accelerators. Pre-aligned floating-point computing-in-memory (FP CIM) has been proposed to achieve high-precision neural network (NN) computations based on floating-point (FP) data precision. However, the complex digital circuitry required for integer (INT) mantissa multiply-accumulate (MAC) computation and exponent alignment severely limits the efficiency and throughput of FP CIM. This work proposes an energy- and area-efficient computing-in-memory (CIM) engine for one-shot FP NN inference and on-device fine-tuning. To improve the throughput of FP CIM, a one-shot compute scheme is proposed to perform FP operation within one cycle. It adopts the multiply-less NN instead of the multiply-based NN to simplify the integer mantissa MAC to minimum selection. A customized 8-bit parallel minimum selector is also designed to further reduce the parallel computation cost. To simplify the FP/INT conversion process, an input–weight co-alignment workflow is proposed to eliminate maximum exponent selection and simplify mantissa shifting logic. To minimize the inference accuracy loss caused by environmental changes, a lightweight on-device fine-tuning core (ODFC) is designed to support online weight updates. The 28-nm fabricated chip achieves an energy efficiency of 128 TFLOPS/W and a computational density of 7.02 TFLOPS/mm^2 at BF16, representing a 4.1× and 3.4× improvement over previous state-of-the-art works, respectively.**

***Index Terms*— Compute-in-memory (CIM), floating-point (FP) operation, on-device fine-tuning, one-shot.**

Received 19 July 2024; revised 24 October 2024 and 10 December 2024; accepted 15 December 2024. Date of publication 7 January 2025; date of current version 27 August 2025. This article was approved by Associate Editor Meng-Fan Chang. This work was supported in part by Beijing Natural Science Foundation under Grant L244051, in part by Beijing Major Science and Technology Project under Grant Z241100004224004, in part by the National Natural Science Foundation of China under Grant 62304009, and in part by the 111 Project under Grant B18001. *(Corresponding author: Xiyuan Tang.)*

Haikang Diao, Haoyang Luo, and Xiyuan Tang are with the Institute for Artificial Intelligence and the School of Integrated Circuits, Peking University, Beijing 100871, China (e-mail: diaohaikang@stu.pku.edu.cn; xitang@pku.edu.cn).

Jiahao Song is with the Institute for Neural Computation (INC), University of California at San Diego, La Jolla, CA 92093 USA.

Bocheng Xu, Runsheng Wang, and Yuan Wang are with the School of Integrated Circuits, Peking University, Beijing 100871, China.

Color versions of one or more figures in this article are available at https://doi.org/10.1109/JSSC.2024.3522304.

Digital Object Identifier 10.1109/JSSC.2024.3522304

I. Introduction

DEEP learning based on neural networks (NNs) has achieved state-of-the-art performance in various tasks [1], [2], [3], [4], stimulating the rapid rise of edge AI. The computing-in-memory (CIM) design, which integrates computational units into memory to directly perform multiply-accumulate (MAC) computation, has proven to be a promising architecture for NN accelerator in edge AI [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16]. It reduces the weight movement and activates multiple rows to perform MAC computation in parallel, dramatically boosting energy efficiency and throughput.

As edge AI applications continue to evolve, they face increasingly complex tasks, such as object detection and on-device model fine-tuning. Therefore, integer (INT) NN models based on integer quantization (e.g., INT8) are insufficient to meet the demands of these complex tasks. More powerful NN models based on floating-point (FP) operations are required. Previous CIM works have focused on combining with NN model quantization to enhance the efficiency of integer operations. To address the complex tasks in edge AI, recent research has extended CIM to high-precision FP operations, further enhancing the performance of NN models [17], [18], [19], [20], [21], [22], [23], [24]. Researchers first explored to implement both mantissa multiplication and exponent addition directly within the CIM array [22], [23], [24]. For example, an all-digital configurable CIM macro is proposed to support the FP8 operation [22]. However, digital circuits for INT mantissa MAC computation and exponent alignment consume 98% of the energy and area, severely limiting the energy efficiency and computational density.

To further reduce the area and energy overhead, pre-aligned FP computing-in-memory (FP CIM) interleaves input alignment and INT mantissa MAC [17], [18], [19], [20], [21]. As illustrated in Fig. 1, four steps are required for the FP CIM. Step 1 ($X_{E\max}$ selection) finds the maximum exponent of the inputs $X_{E\max}$ using multi-level comparators. Then, in step 2 (Input alignment), the mantissa of the inputs is aligned to $X_{E\max}$, which is used for the integer mantissa MAC with the weights in step 3 (mantissa MAC). Finally,

0018-9200 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:23:27 UTC from IEEE Xplore. Restrictions apply.

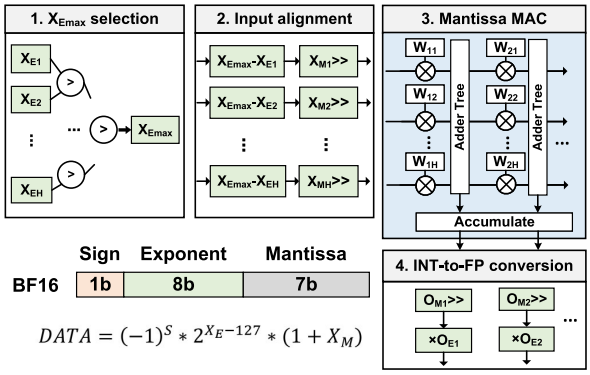


Fig. 1. Typical architecture of the pre-aligned FP CIM.

the integer MAC results are normalized into the FP format in step 4 (INT-to-FP conversion). By pre-aligning the mantissa in advance, it eliminates the need for complex alignment and shifting operations within the CIM array, thereby improving the efficiency of FP operation.

However, as shown in Fig. 2, prior pre-aligned FP CIM still faces several major challenges.

1) The INT mantissa MAC severely limits the throughput of FP CIM. In current FP CIM works, steps 1, 2, and 4 each require only one cycle, while step 3 usually uses bit-serial multiplication, necessitating multiple cycles to complete the integer mantissa MAC computation. For example, for the BF16 data precision, step 3 requires eight cycles for the INT8 mantissa MAC computation. As a result, circuits designed for steps 1, 2, and 4 will be idle for 87.5% of the time, significantly reducing hardware utilization and throughput. Although Booth encoding is used in [17] to reduce cycle count by sending 2 bits per cycle, it requires additional circuitry for data encoding and decoding and still takes four cycles to achieve BF16 computation. If bit-parallel multiplication is used in step 3, the multiplier and adder tree consume over 96% overall area and energy.
2) The FP and integer conversion incurs additional area and energy costs. For example, when the data precision is BF16 and the input dimension is 128, step 1 requires 127 8-bit comparators to find the maximum exponent. Step 4 searches the 16-bit integer output to find the highest bit "1," and then performs truncation and shifting for FP format conversion. Both the steps introduce a significant combinational logic delay and increase area and energy costs.
3) The environmental changes decrease the NN model accuracy. Edge devices frequently face changing environmental conditions (e.g., lighting, weather), leading to differences between test samples and training set samples. Previous FP CIM works do not support on-chip fine-tuning, resulting in decreased inference accuracy under varying environmental conditions. As shown in Fig. 2, when training the weights on MNIST and testing on MNIST-M to simulate background changes in real scenes, the inference accuracy drastically drops from 99.1% to 13.8%.

To overcome the above challenges, this work proposes an energy- and area-efficient CIM engine for one-shot pipelined FP NN inference and on-device fine-tuning. It leverages the joint optimization of algorithms, architecture, and circuits to complete complex FP operation within one cycle efficiently. The primary contributions of this work are listed as follows.

1) This work proposes a one-shot compute scheme with a parallel minimum selector to perform FP operation within one cycle. It adopts a multiply-less NN instead of the multiply-based NN that simplifies the INT mantissa MAC to minimum selection. The parallel minimum selector is proposed to reduce the area and energy consumption of parallel minimal selection.
2) This work proposes an input–weight co-alignment workflow to simplify the FP/INT conversion process. It directly uses the maximum exponent of the weights to align the input mantissa, thereby eliminating the max exponent selection in step 1. Meanwhile, based on the non-negative characteristics of the multiply-less NN, it simplifies the mantissa shifting process in step 4.
3) This work proposes a lightweight on-device fine-tuning core (ODFC). It can perform online model updates in response to edge-side environmental changes, thereby ensuring accuracy.

This article is an extension of [19] and is organized as follows. Section II describes the proposed architecture of the CIM engine. Section III presents the one-shot compute scheme and the circuit details of the parallel minimum selector. Section IV describes the input–weight co-alignment workflow for FP/INT conversion. Section V presents the architecture and data flow of the ODFC. Section VI presents the prototype measurements and NN demonstrations. The conclusion is drawn in Section VII.

II. Proposed Architecture of the CIM Engine

The overall architecture of the CIM engine is shown in Fig. 3. It comprises a mantissa alignment unit (ManAU) for core-wise input–weight co-alignment, a CIM core for INT mantissa computation, an INT-to-FP unit (ITFU) that converts the integer output back into FP format, and an ODFC for online model update. The CIM core contains an input driver, an input adder tree (InADD), and 16 columns of subarrays for 16-kb NN weight storage. Each subarray contains 128 8-bit 6T SRAM cells for NN weights storage, 15-bit 6T SRAM for weights summation (WSUM) storage, 128 parallel minimum selectors (ParMSs), and an accumulate–assembly unit (ACC–ASU).

It is worth noting that the proposed CIM macro supports both FP and INT computations, both requiring only one cycle. In the FP mode, the ManAU shifts the input's mantissa to align its exponent with the weights' maximum exponent. The aligned input's mantissa X_{MA} is sent to the CIM core for INT mantissa computation with weight. The CIM core outputs are received by the ITFU and converted back into FP format. In the INT mode, inputs and outputs are computed directly without the involvement of the ManAU and ITFU.

Inspired by [25], we adopt multiply-less NN to replace multiply-based NN, eliminating costly multiplication

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:23:27 UTC from IEEE Xplore. Restrictions apply.

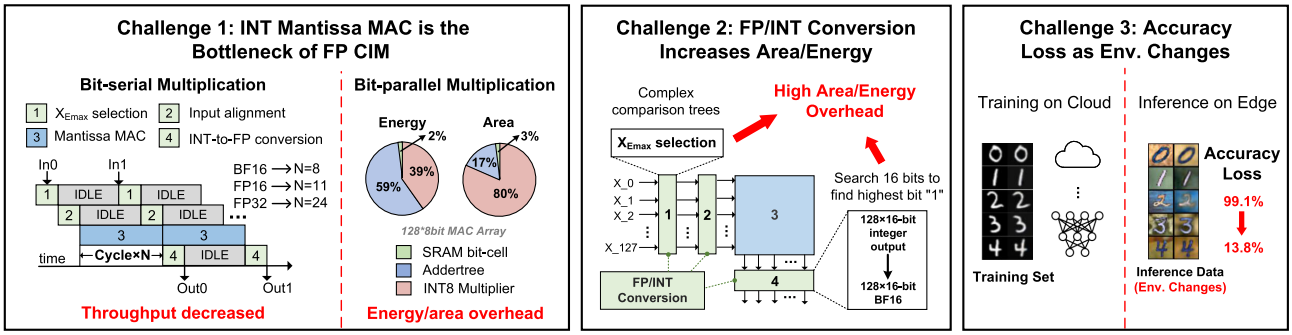


Fig. 2. Three challenges of prior pre-aligned FP CIM.

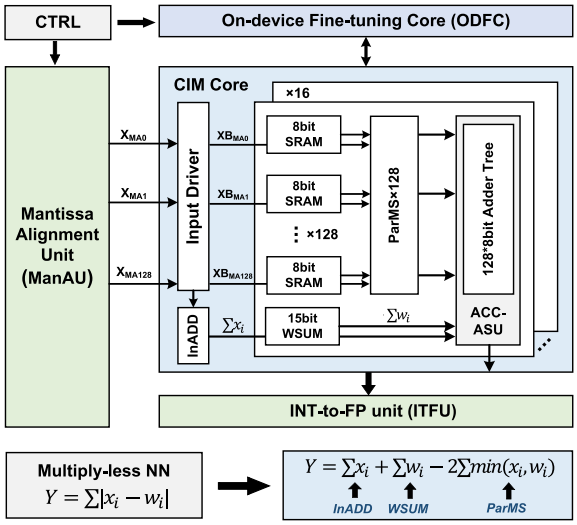


Fig. 3. Overall architecture of the CIM engine.

TABLE I
PERFORMANCE COMPARISON OF MULTIPLY-LESS NN AND MULTIPLY-BASED NN

Task	Model		Dataset	Performance	
				Multiply-based	Multiply-less
Image Classification	CNN [25]	ResNet-20	CIFAR-100	68.1%	67.6%
		ResNet-50	ImageNet	76.2%	74.9%
	Transformer [27]	DeiT-B		81.8%	80.4%
		DeiT-S		79.9%	78.3%
Object Detection	R-CNN [28]		COCO	37.9% (mAP)	37.0% (mAP)
Super Resolution	VDSR [29]		B100	31.9 (PSNR)	31.8 (PSNR)
Image Compression	NTCF [30]		Kodak	33.8 (PSNR)	31.9 (PSNR)

operations to simplify integer mantissa MAC computation. As shown in the bottom of Fig. 3, the multiply-less NN uses $L1$ distance instead of the dot product operation between weights and inputs. A theoretical analysis in [26] proves that multiply-less NN has the same expressive power as multiply-based NN. Table I shows the performance of the multiply-less NN across different models (e.g., CNN and transformer) and various tasks (e.g., object detection and image compression) [25], [27], [28], [29], [30], [31]. The results indicate that multiply-less NN achieves comparable performance and scalability to multiply-based NN. Moreover, similar to the approach in [14], we expand the $L1$-distance operation and convert it into a minimum selection operation to further reduce computational complexity. As shown in

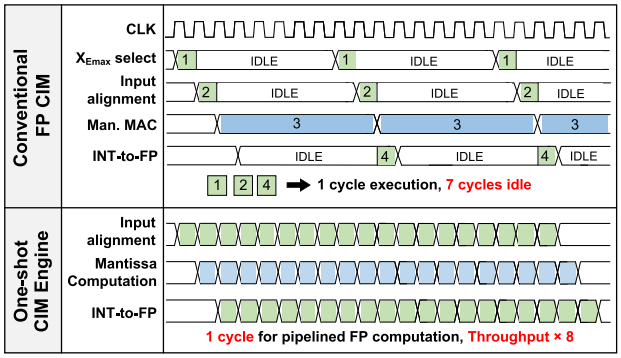


Fig. 4. Comparison between the conventional FP CIM computation scheme and the one-shot computation scheme in the BF16 data format.

Fig. 3, the sum of inputs x_i is calculated by InADD, which is implemented as a 128 8-bit adder tree. The result of InADD is shared across the entire CIM core to amortize the extra computation costs. The sum of weights w_i is precomputed before model deployment and stored in WSUM, and the minimum selection and summation $2\sum \min(x_i, w_i)$ are performed by ParMSs and ACC–ASU. This article combines the multiply-less NN and computing-in-memory architecture to alleviate the memory bottleneck. Thus, the performance and efficiency of FP NN inference computation are greatly improved.

III. ONE-SHOT COMPUTE SCHEME FOR FP OPERATION

A. One-Shot Compute Scheme

Fig. 4 illustrates the comparison between the conventional FP CIM compute scheme and the one-shot compute scheme in the BF16 data format. In the conventional FP CIM compute scheme, step 1 of $X_{E\max}$ selection, step 2 of input alignment, and step 4 of INT-to-FP conversion can be completed within one cycle. However, to reduce the MAC computation cost, step 3 usually adopts a bit-serial data flow. It splits the 8-bit mantissa and only inputs 1 bit per cycle to the CIM core for MAC computation with the weights. Therefore, a total of eight cycles are required in step 3. As a result, the circuits for steps 1, 2, and 4 are idle for seven cycles, reducing hardware utilization and throughput. In the one-shot compute scheme, the INT8 mantissa computation is completed within one cycle. It is enabled by the simplicity of minimum selection in multiply-less NN, resulting in significantly lower hardware overhead for bit-parallel data flow. As a result, input alignment, mantissa computation, and INT-to-FP conversion each take

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:23:27 UTC from IEEE Xplore. Restrictions apply.

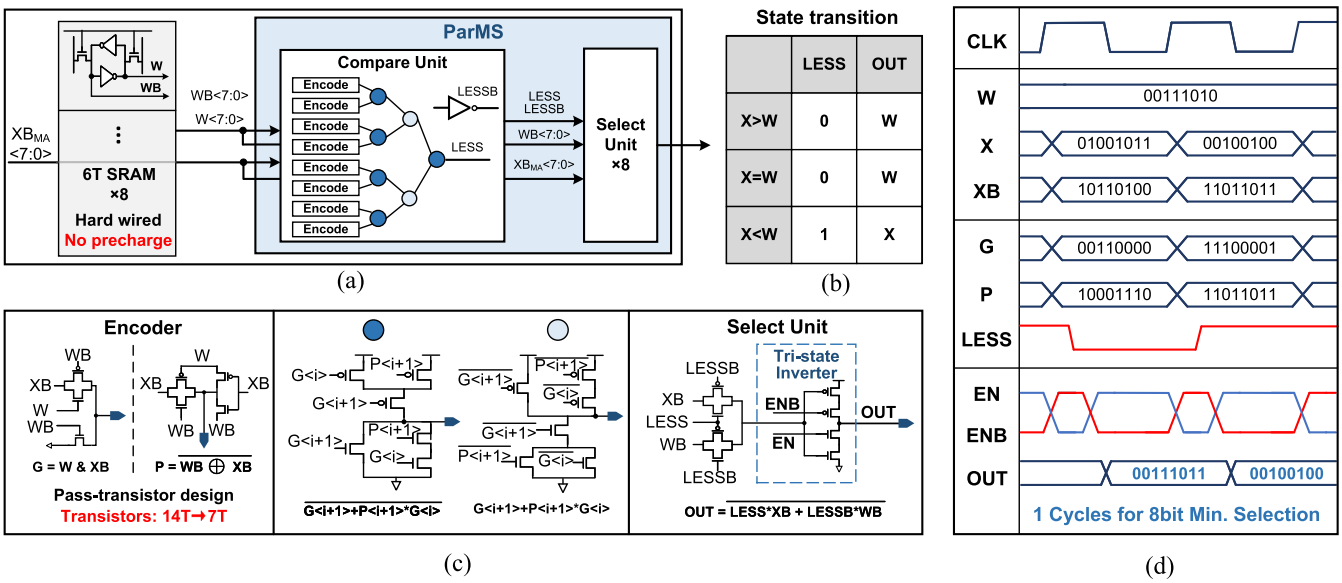


Fig. 5. (a) Overall architecture of the ParMS, (b) state transition for the ParMS, (c) circuit details of the ParMS (the group P signals' generation are not shown), and (d) waveform for ParMS.

only one cycle, achieving pipelined FP computation. That leads to an 8× throughput increase.

B. Parallel Minimum Selector Design

Although the multiply-less NN eliminates the integer MAC computation, the required 8-bit minimum selection still incurs high area and power costs. To further increase the efficiency, a customized 8-bit parallel minimum selector (ParMS) is designed, as shown in Fig. 5. It consists of a comparison unit and eight select units. To avoid the power overhead of pre-charge for SRAM, a hard-wired design is adopted in ParMS. The Q node and QB node of the SRAM bit-cell are directly connected to the ParMS. To reduce its combinational logic delay, a carry-lookahead design is adopted in the comparison units. Each comparison unit comprises eight encoders and a comparison tree. The eight encoders convert the 8-bit $\mathrm{XB_{MA}}$ (the inverse of the mantissa of the input X), W, and WB in parallel into 8-bit generate signals (G) and 8-bit propagate signals (P)

$$\begin{aligned} G\langle i\rangle &= W\langle i\rangle \ \&\ \mathrm{XB}\langle i\rangle \\ P\langle i\rangle &= \mathrm{WB}\langle i\rangle \bigoplus \mathrm{XB}\langle i\rangle. \end{aligned} \tag{1}$$

Then the comparison tree outputs the comparison results' signal (LESS)

$$\mathrm{LESS} = G\langle 7\rangle + \sum_{k=0}^{6}\left(G\langle k\rangle \ \&\ \prod_{m=k+1}^{7} P\langle m\rangle\right). \tag{2}$$

Finally, the select unit outputs the smaller value between W and X based on the LESS signal

$$\mathrm{OUT} = \overline{\mathrm{LESS}\times \mathrm{XB}\ + \mathrm{LESSB}\times \mathrm{WB}}. \tag{3}$$

Fig. 5(c) shows the circuit details of the proposed ParMS. The Encoder adopts a pass transistor logic design to minimize area and power, reducing the number of transistors from 14 to seven. To ensure signal integrity, we adopted a one-stage pass-transmission gate design in the encoder. In addition, the transmission gate consists of a symmetric NMOS and PMOS pair, effectively mitigating threshold voltage loss and maintaining signal quality. The digital logic of the comparison tree is relatively complex. We expanded the logic in (2) to simplify its circuit implementation. First, we define group G and P signals as follows:

$$\begin{aligned} G\langle 2i+1{:}2i\rangle &= G\langle 2i+1\rangle + P\langle 2i+1\rangle * G\langle 2i\rangle \\ P\langle 2i+1{:}2i\rangle &= P\langle 2i+1\rangle * P\langle 2i\rangle \\ G\langle 4j+3{:}4j\rangle &= G\langle 4j+3{:}4j+2\rangle \\ &\quad + P\langle 4j+3{:}4j+2\rangle * G\langle 4j+1{:}4j\rangle \\ P\langle 4j+3{:}4j\rangle &= P\langle 4j+3{:}4j+2\rangle * P\langle 4j+1{:}4j\rangle \end{aligned} \tag{4}$$

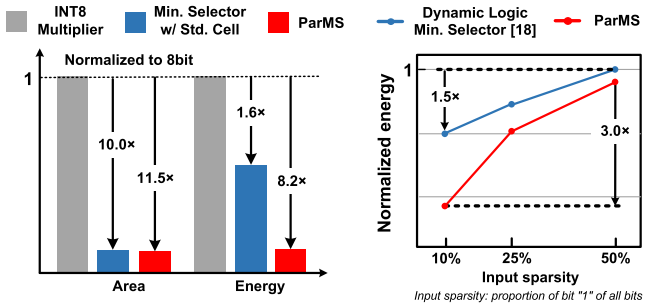
where $i = 0, 1, 2, 3$ and $j = 0, 1$. Then, (2) can be further expanded as

$$\mathrm{LESS} = G\langle 7{:}4\rangle + P\langle 7{:}4\rangle G\langle 3{:}0\rangle. \tag{5}$$

Equation (5) represents a three-level nested digital logic, with each level having the same simple logic as group G signal in (4). Therefore, we use a three-stage circuit to implement the comparison tree as shown in Fig. 5(c). The logic of adjacent stages is opposite, thus eliminating the need for inverters. This tree-structured circuit significantly reduces the complexity of the logic and the number of transistors, thereby reducing area and power consumption.

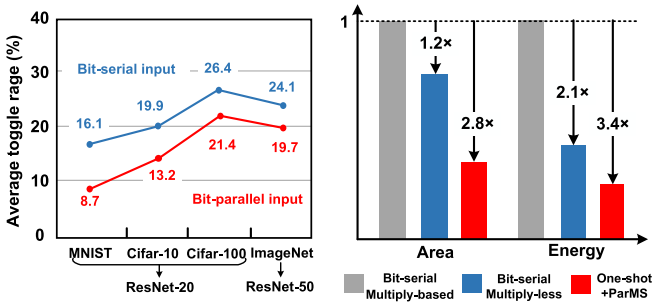
The selection unit consists of two transmission gates and a tri-state inverter controlled by EN and ENB signals. The EN and ENB signals are used to gate the output of the ParMS to reduce the dynamic power consumption of the adder tree caused by unnecessary flipping from the ParMS. As shown in Fig. 5(d), setting EN = 0 and ENB = 1 disables the ParMS output at the rising edge of the clock. When the ParMS completes its computation and stabilizes, setting EN = 1 and ENB = 0 transmits the ParMS output to the adder tree. Using this tri-state inverter, the power consumption of the adder tree can be reduced by 44%. In addition, we designed a configurable delay generation model in the chip that can adjust

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:23:27 UTC from IEEE Xplore. Restrictions apply.



(a) (b)

Fig. 6. Evaluation for the parallel minimum selector (a) area and energy comparison and (b) energy comparison at different input sparsity between the dynamic-logic-based minimum selector in [14] and ParMS.



(a) (b)

Fig. 7. (a) Average toggle rates for different datasets and NN models and (b) performance evaluation for one-shot compute scheme.

the delay time to accommodate different operating frequencies and voltages. Due to the low combinational logic delays of the ParMS, the OUT node is floated for less than 1/8 clock cycle. Therefore, the voltage drop at the OUT node is minimal and does not increase the power consumption of the back-end circuit.

C. Technique Evaluation

Fig. 6 shows the performance evaluation for the ParMS. Compared with an INT8 multiplier, ParMS reduces area by 11.5× and energy by 8.2×, respectively. When compared with digital comparators generated using the digital synthesis flow, ParMS reduces area by 14% and energy by 4.1×, respectively. NNs typically exhibit high sparsity, especially when the ReLU function truncates all negative inputs to 0. Therefore, ParMS adopts a fully static logic design to leverage the NN sparsity. As shown in Fig. 6(b), compared with the dynamic logic-based comparator proposed in [14], ParMS demonstrates a significant energy reduction as sparsity increases. The custom design of ParMS enables the proposed CIM engine to perform INT8 mantissa computation more efficiently.

The bit-parallel computation used in the one-shot compute scheme can exploit the input data correlation to reduce the toggle rate. In NNs, adjacent input data are often similar to each other. Specifically, the MSBs are more likely to be the same across adjacent input data, whereas the LSBs vary more. As a result, the MSBs feature a lower toggle rate than LSBs. Fig. 7(a) shows the average input data toggle rates after input alignment for different datasets and NN models. Bit-parallel computation can leverage this data correlation to reduce the

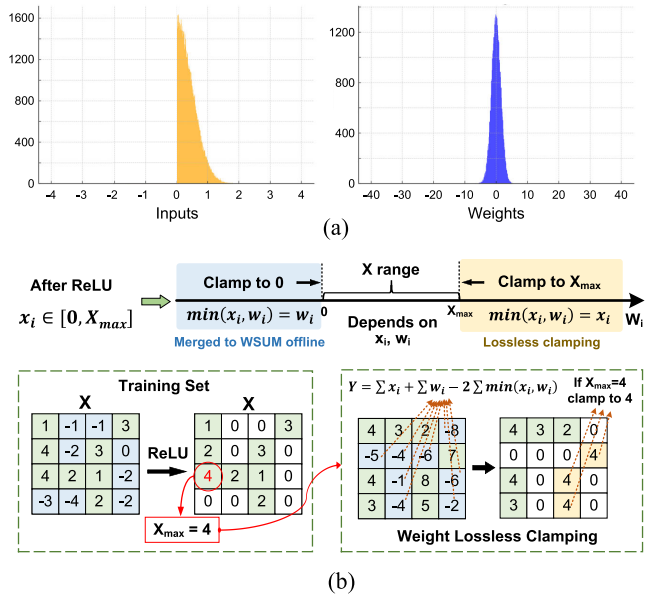


Fig. 8. (a) Range of inputs and weights and (b) offline weight lossless clamping.

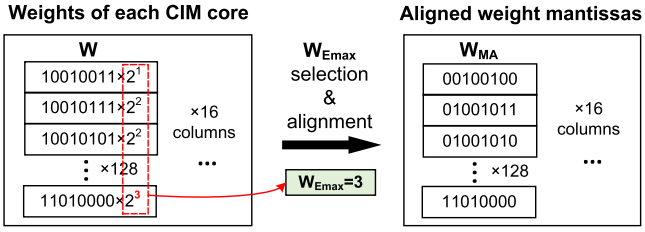


Fig. 9. Example of the offline weight pre-alignment.

average toggle rate, thereby lowering power consumption. In contrast, the traditional FP CIM, which uses bit-serial MAC from MSBs to LSBs, cannot take advantage of this data correlation. Fig. 7(b) shows the performance evaluation for the proposed one-shot compute scheme. The area and energy results of bit-serial multiply-based and bit-serial multiply-less are based on post-synthesis simulation, evaluated at 28 nm, 0.9 V, 180 MHz. Compared with the conventional multiply-based FP CIM, the one-shot CIM engine proposed in this work reduces area by 2.8× and energy by 3.4×.

IV. Input–Weight Co-Alignment Workflow

For complex FP operations, as indicated by the second challenge in Fig. 2, the FP/integer conversions in steps 1 and 4 increase area and energy consumption. To address this issue, this work leverages the data distribution characteristics of multiply-less NN to achieve input–weight co-alignment workflow. Specifically, an offline weight lossless clamping is performed, ensuring that the weights and inputs have the same data distribution range. During the inference, the weights' local maximum exponent of each CIM core can be directly used to align the inputs, thus eliminating step 1 ($X_{E\max}$ selection). In addition, after processing through the ReLU function, the inputs have no negative values. Similarly, the weights, after lossless truncation, also have no negative values. As a result, the outputs of the CIM core are non-negative. Therefore, we propose the unsigned BF16 format (UBF16)

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:23:27 UTC from IEEE Xplore. Restrictions apply.

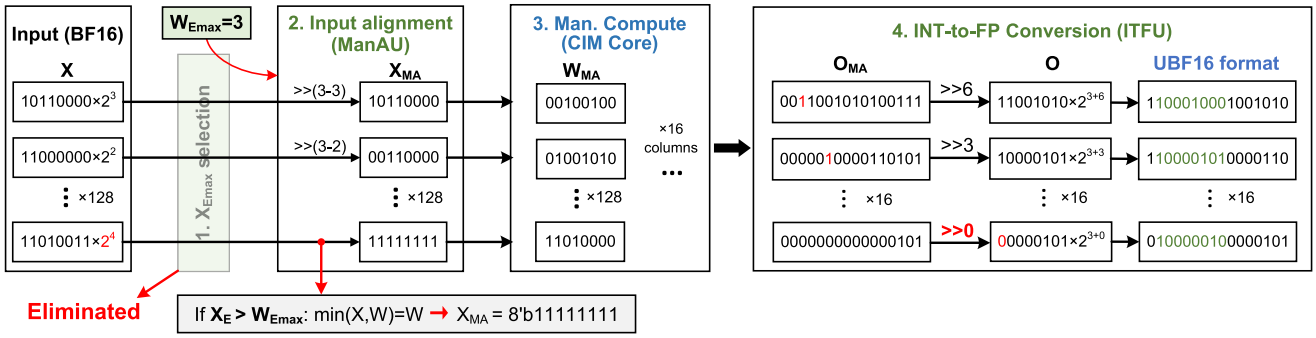


Fig. 10. Online input–weight co-alignment pipeline.

to simplify the integer normalization process, which will be described in detail in Section IV-B.

A. Offline Weight Lossless Clamping and Pre-Alignment

In the multiply-less NN, as shown in Fig. 8(a), the range of weights is larger than that of inputs. Since the multiply-less NN operator is the minimum selection, we can leverage this data distribution characteristic to pre-determine the results. Fig. 8(b) illustrates the process of offline weight lossless clamping. After the ReLU activation function, the input X is clamped to [0, $X_{\max}$], where $X_{\max}$ is an empirical value obtained from the training set. For weight less than 0, as the X is non-negative, it can be pre-determined that the minimum value would be the weight without the need for computation. Therefore, negative weights can be directly clamped to 0, and combined as a bias in advance. For weight greater than $X_{\max}$, since all inputs do not exceed $X_{\max}$, it can be pre-determined that the minimum value is $X_{\max}$. Thus, weights greater than $X_{\max}$ can be clamped to $X_{\max}$. It is worth noting that during the actual inference process, the maximum value of the input, X_{infmax}, varies with different samples. However, 95% of X_{infmax} are less than $X_{\max}$, which is obtained from the training set, so the inference accuracy is hardly affected. After lossless clamping, the weight distribution is aligned with the input X ([0, $X_{\max}$]). Therefore, we can use the maximum exponent of the weights to directly align the mantissa of the inputs without introducing significant errors. Moreover, lossless clamping increases the sparsity of the weights, thereby reducing energy.

Fig. 9 shows a simple example of offline weight pre-alignment. For the weights of each CIM core, the maximum exponent $W_{E\max}$ is selected and stored in advance. The mantissa of weights is then pre-aligned based on $W_{E\max}$ and stored in the CIM core.

B. Online Input–Weight Co-Alignment

Fig. 10 shows the online input–weight co-alignment workflow. The inputs' mantissa is directly aligned by $W_{E\max}$ in the ManAU, eliminating step 1 ($X_{E\max}$ selection) in the conventional FP CIM. These aligned mantissas X_{MA} are then sent to the CIM core to perform the integer mantissa computations with the 16 columns of pre-aligned weights' mantissa W_{MA}. Finally, the ITFU converts the 16-bit integer results O_{MA} back into an FP format. Note that in the test

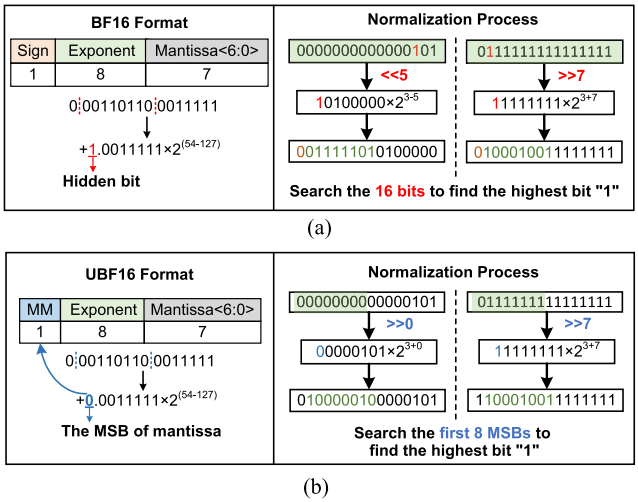


Fig. 11. (a) BF16 format and normalization process and (b) UBF16 format and normalization process.

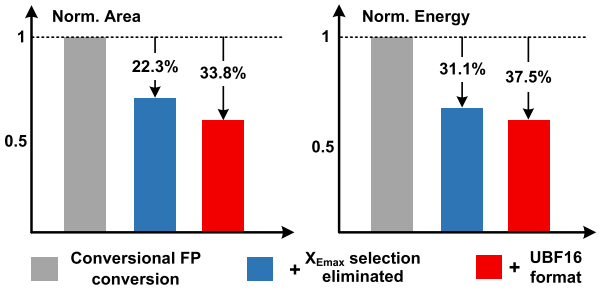


Fig. 12. Performance evaluation for input–weight co-alignment workflow.

set, there may be cases where the input exponent is greater than $W_{E\max}$. Since the computation in the CIM Core is to find the minimum value, the result will be the weight in this case. The ManAU directly sets the input's mantissa to the maximum value (8′11111111) to ensure that the minimum value is weight.

For the convention FP CIM, the final step (INT-to-FP conversion) is to convert integer data into BF16 format. As shown in Fig. 11(a), the mantissa is 8 bit-width and its MSB must be "1," which is known as the hidden bit. Seven lower bits of the mantissa are stored in BF16 format (Mantissa⟨6:0⟩). Therefore, normalizing a 16-bit integer result requires searching through 16 bits to find the highest bit "1" and then shifting the mantissa accordingly. This increases the area and delay overhead.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:23:27 UTC from IEEE Xplore. Restrictions apply.

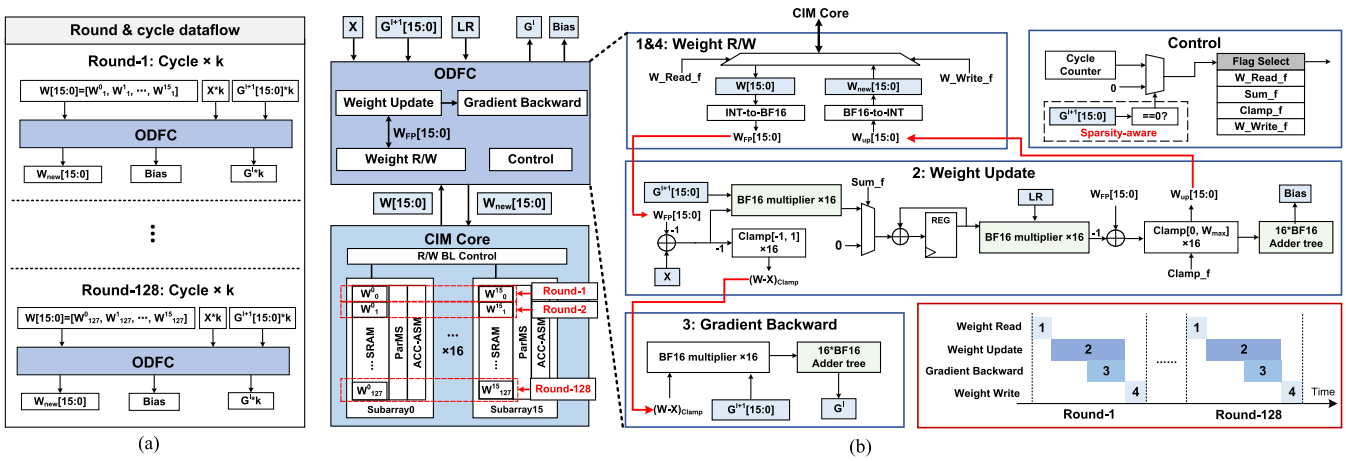


Fig. 13. (a) Round and cycle data flow for on-device fine-tuning and (b) overall architecture of the ODFC.

For our proposed workflow, since X_{MA} and W_{MA} are non-negative, O_{MA} from selection and summation would also be non-negative. Thus, the unsigned BF16 (UBF16) format is proposed in this design as shown in Fig. 11(b). In UBF16, the highest bit represents the mantissa MSB instead of the sign bit in BF16. Unlike the hidden bit in BF16, the mantissa MSB in UBF16 may not be "1." Thus, the search range for the highest bit "1" in O_{MA} is shortened from 16 to 8. If no bit "1" is found in the higher 8 bits, the lower 8 bits can be directly stored as the mantissa with MSB stored in the highest bit of UBF16. Therefore, by introducing the UBF16 format, we simplify the process of searching for the highest bit "1," thereby reducing the area and delay of digital logic circuits. When UBF16 data are subsequently computed with BF16 data, the highest bit of the UBF16 is directly combined with Mantissa⟨6:0⟩ to form the 8-bit mantissa, allowing for normal FP operation. This process is very simple and incurs minimal hardware overhead.

C. Technique Evaluation

Fig. 12 shows the evaluation of the pre-aligned workflow. The baseline is the conventional FP CIM with 128 BF16 inputs, and the measured area and energy include the digital logic of steps 1, 2, and 4. The baseline results are based on post-synthesis simulation and evaluated at 28 nm, 1 V, 180 MHz, and 10% input sparsity. By eliminating step 1 through offline weight lossless clamping, the area and energy can be reduced by 22.3% and 31.7%, respectively. By introducing the UBF16 data format to simplify step 4, the area and energy can be further reduced by 11.5% and 6.5%, respectively.

V. On-Device Fine-Tuning

The architecture of the on-device fine-tuning core and training algorithms for multiply-less NN are shown in Figs. 13 and 14. Compared with the minimum value selection operation for inference, the training algorithm has significantly higher computational complexity. It requires much more complex circuits than inference to handle extensive computations. However, the cost constraints in edge intelligence restrict the chip's area. Moreover, edge-side fine-tuning requires significantly fewer training epochs than training from scratch.

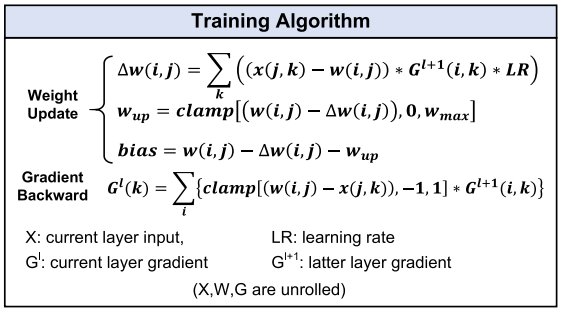


Fig. 14. Training algorithm for the multiply-less NN.

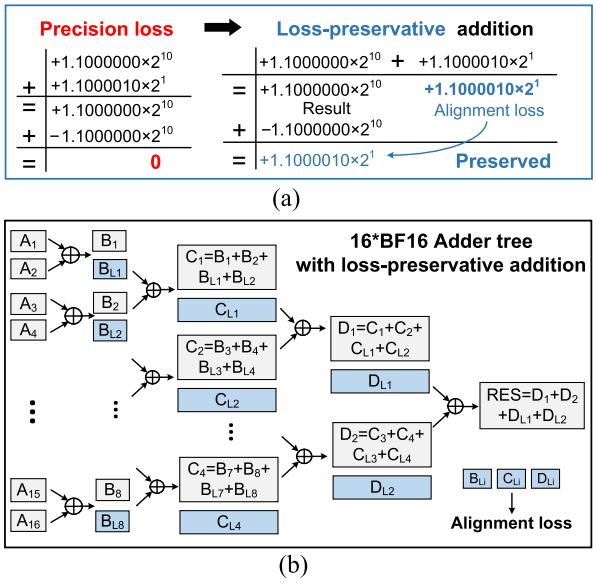
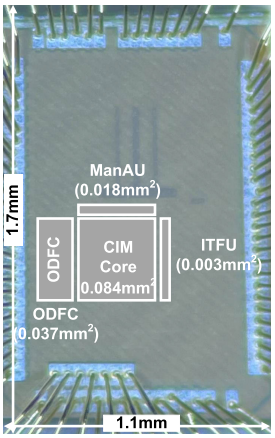


Fig. 15. (a) Loss-preservative FP addition and (b) 16 ∗ BF16 adder tree.

Therefore, we propose a lightweight ODFC with multi-round data flow to perform fine-tuning in an area manner. It reduces the scale of parallel computations and alleviates the ODFC's area cost.

Fig. 13(a) shows the round and cycle data flow, and Fig. 13(b) shows the architecture design of ODFC. The round and cycle data flow takes 128 rounds to sequentially update the 128 × 16 weights in the entire CIM core. Each round consists of four phases: weight read, weight update,

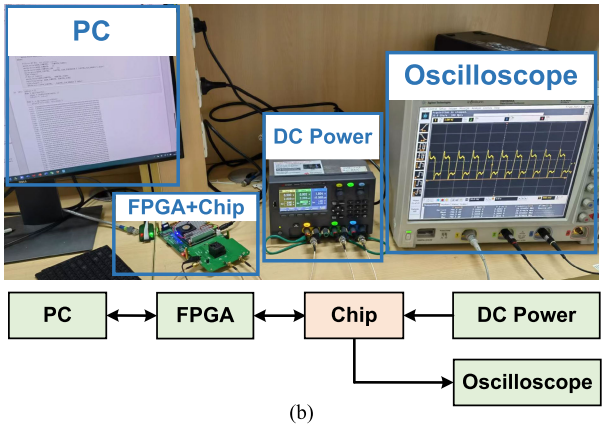
Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:23:27 UTC from IEEE Xplore. Restrictions apply.



Chip Summary		
Technology	28nm	
Macro Area	0.143mm²	
SRAM Capacity	16Kb (logic)	
Bitcell Area	0.481um²	
Voltage	0.55-0.9V	
Frequency	20-180MHz	
Compute Function	Min. SEL+ADD	
Compute Delay[a]	1.87ns	
Throughput[c]	737GOPS (180MHz)	
Output Ratio	0.5 (16bit-to-8bit)	
Data Precision	INT8	UBF16
Energy Efficiency	58[c]-152[b] TOPS/W	49[c]-128[b] TFLOPS/W
Area Efficiency[c]	8.76 TOPS/mm²	7.02 TFLOPS/mm²

[a]Estimated by postlayout simulation at 0.9V, 180MHz, 25℃, 50% input and weight sparsity.
[b]Measured at 0.55V, 20MHz, 25℃, 10% input sparsity and 50% weight sparsity, ODFC core is not included.
[c]Measured at 0.9V, 180MHz, 25℃, 10% input sparsity and 50% weight sparsity, ODFC core is not included.

(a)



(b)

Fig. 16. (a) Chip micrograph and (b) test platform.

gradient computation, and weight write. First, the mantissas of 16 weights are read from the CIM core and normalized to BF16 format. Then, the ODFC computation is performed within k cycles, where k is the length of the gradient vector in the training algorithm. In each cycle, ODFC obtains $1\times$ and 16 gradients from the latter layer and updates the fetched weights and gradients for the previous layer. After k cycles, the updated weights and biases are computed. Finally, the updated weights are aligned, and the mantissas are written back to the CIM core. Since the ODFC only updates 16 weights each round, the circuit area is significantly reduced. In addition, we time-multiplex the BF16 multipliers and BF16 adder trees to share hardware across different phases of the computation, further reducing the circuit area by 10%.

When multiple FP numbers are accumulated, intermediate precision loss can occur due to shift alignment. As shown in Fig. 15(a), when a large number is added to a small number, the mantissa of the small number may become 0 after shift alignment, resulting in the sum of three numbers being 0. To reduce the alignment loss, we propose a loss-preservative FP addition. The error caused by the alignment is retained and added in the next level of addition. Furthermore, as shown in Fig. 15(b), we use the loss-preservative FP addition to the $16*$BF16 adder tree to reduce the alignment loss in multilevel FP summation.

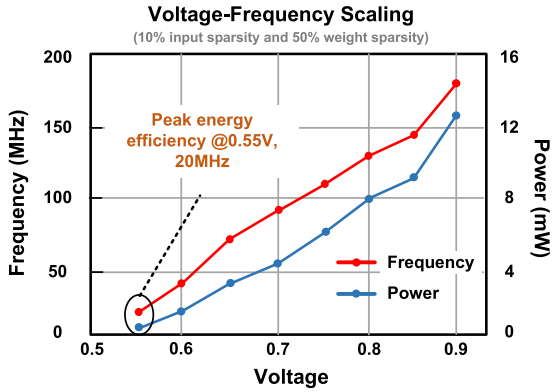


Fig. 17. Voltage–frequency scaling.

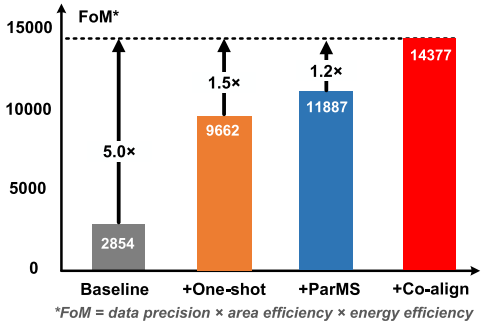


Fig. 18. Breakdown of the improvements. Baseline: a 28-nm FP multiply-less CIM macro based on [14].

VI. Measurement Results

A. Chip Micrograph and Summary

Fig. 16(a) presents the die photograph and summarizes the chip measurement results. This chip is fabricated in 28-nm CMOS technology with a 1.7 × 1.1 mm area. The chip integrates a CIM core (0.084 mm^2), an ManAU (0.018 mm^2), an ITFU (0.003 mm^2) for BF16 inference, and an ODFC (0.037 mm^2) for on-chip fine-tuning. Unlike conventional FP CIM's multiply-accumulate operations, the computational functions of this chip are minimum selection and accumulation. Therefore, the calculation of throughput is different from the dot-product-based NN chips. In our chip, a pair of minimum selections and accumulation of 1 input and 1 weight are counted as 2 Ops. The voltage scaling of this chip is measured from 0.55 to 0.9 V, with the operating frequency from 20 to 180 MHz. To ensure the normal operation of the SRAM in the CIM, the read and write operations of the SRAM are only performed at 0.9 V. For the low-voltage test, we first maintain the voltage at 0.9 V to write the weights into the SRAM, and then reduce the supply voltage to perform the computation. At low voltage, the SRAM only needs to hold the output signal statically, thereby enabling a wider operating voltage range. The measured peak performance for UBF16 is 737GOPS at 0.9 V, 180 MHz. The measured peak energy efficiency for UBF16 is 128 TFLOPS/W at 0.55 V, 20 MHz, 10% input sparsity, and 50% weight sparsity. The measured peak area efficiency for UBF16 is 7.02 TFLOPS/mm^2 at 0.9 V, 180 MHz.

Fig. 16(b) describes the test platform for the fabricated chip, including the CIM engine chip, an FPGA board, a dc power,

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:23:27 UTC from IEEE Xplore. Restrictions apply.

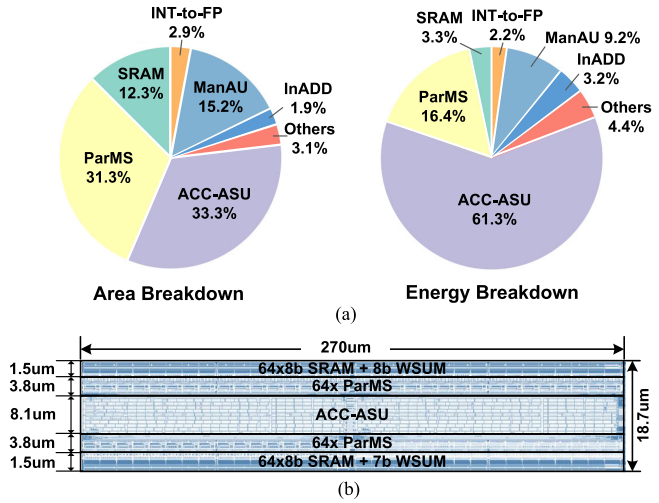


Fig. 19. (a) Energy and area breakdown of test-chip and (b) layout of one column in the CIM core (SRAM, WSUM, ParMS, and ACC–ASU).

an oscilloscope, and a host PC. The computer uses Vivado to generate bitstream and program it into the FPGA board to set up the test platform. A customized compiler on the PC side generates instructions and data for chip configuration and execution.

As shown in Fig. 17, the voltage scaling of the chip is measured. The peak energy efficiency for UBF16 is 128 TFLOPS/W, running at 0.55 V, 180 MHz. Fig. 18 evaluates the effectiveness of the proposed techniques. FoM is defined as the product of data precision, area efficiency, and energy efficiency. The comparison baseline is a 28-nm FP multiply-less CIM chip, with the architecture based on the design from [14]. This chip uses bit-serial dynamic-logic-based comparator circuits for minimum selection. The comparison baseline results are based on post-synthesis simulation, evaluated at 0.9 V, 180 MHz, 10% input sparsity, and 50% weight sparsity. Compared with the baseline, this work achieves a 5× increase in FoM.

Fig. 19(a) shows the area and energy breakdown of the chip at 0.9 V, 10% input sparsity, and 50% weight sparsity, excluding the ODFC. Based on the input–weight co-alignment workflow, the area and energy proportions of the ManAU and ITFU for FP/INT conversion are only 18.1% and 11.4%, respectively. Fig. 19(b) shows the layout of one column in the CIM core. Compared with FP CIM which uses parallel FP multiplication [22], where the area of the multipliers and adder trees is 49× that of the SRAM, our proposed one-shot compute scheme with parallel minimum selector significantly reduces the area of this digital logic, making it only 5.2× that of the SRAM.

B. Evaluation on Multiply-Less NN

1) NN Model Deployment Flow: The macro architecture is co-designed with software mapping algorithms. Fig. 20(a) shows the custom off-line model deployment flow. First, an FP32 multiply-less pre-trained model is obtained using the standard training flow by PyTorch. Second, the weights and inputs are truncated to BF16 format. Third, the weights are losslessly clamped to [0, $X_{\max}$] and the negative weights are

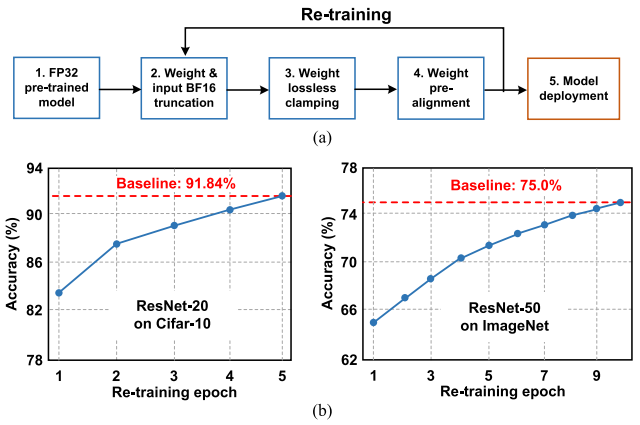


Fig. 20. (a) NN model deployment flow and (b) inference accuracy improvement by model re-training.

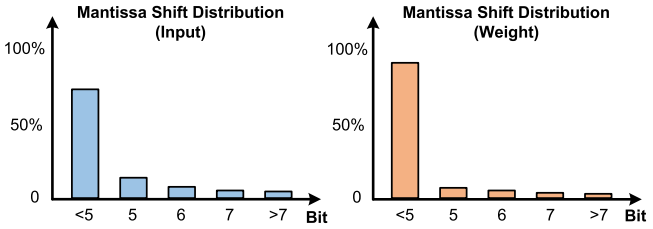


Fig. 21. Mantissa shift distribution of inputs and weights.

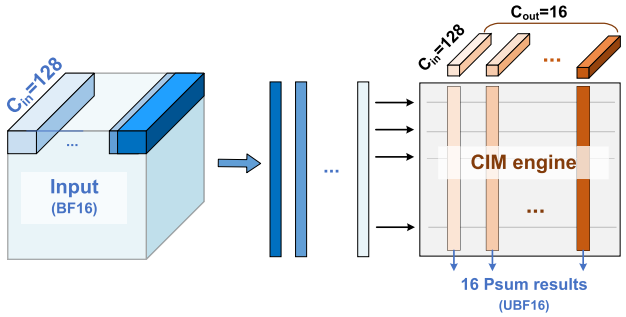


Fig. 22. Layer-by-layer NN mapping method.

merged into the WSUM, as mentioned in Section IV-A. Fourth, the weights mapped to the same CIM core will find their local maximum exponent $W_{E\max}$ and pre-aligned. To mitigate the precision loss caused by pre-alignment, a model re-training process is proposed. It retrains the model with BF16 data precision, followed by reapplying weight lossless clamping and pre-alignment. Note that during the model retraining process, the input must also be aligned using the input–weight co-alignment workflow as mentioned in Section IV-B. Finally, the model is deployed when the model's accuracy is comparable to that of the FP32 model. Fig. 20(b) presents the improvement in inference accuracy through model re-training. After re-training ResNet-20 for five epochs and ResNet-50 for ten epochs, the precision loss caused by pre-alignment is fully compensated, resulting in inference accuracy equivalent to the baseline.

Fig. 21 illustrates the mantissa shift distribution of inputs and weights for the ResNet-20 style multiply-less model inference. About 74.3% of the input mantissas and 89% of the weight mantissas have a shift range of fewer than 5 bits. This indicates that due to the model retraining process, the

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:23:27 UTC from IEEE Xplore. Restrictions apply.

TABLE II
NN DEMONSTRATIONS

Task	Image Classification		Object Detection
Dataset	Cifar-10	ImageNet	COCO
Neural-Network Style	ResNet-20	ResNet-50	FCOS
Bit Precision	UBF16	UBF16	UBF16
Top-1 Accuracy	91.84%	75.00%	36.80% (mAP)
Baseline (FP32)	91.84%	75.00%	37.00% (mAP)

TABLE III
PERFORMANCE OF THE ODFC

Area (mm^2)	Frequency (MHz)	Throughput (GFLOPS)	Area Eff. (TFLOPS/mm^2)	Energy Eff. (TFLOPS/W)*
0.037	180@0.9V	2.88@BF16	0.078@BF16	1.71@BF16

*At 0.9V, 180MHz, 25°C. Energy is estimated by post-layout simulation. Off-chip power is not included.

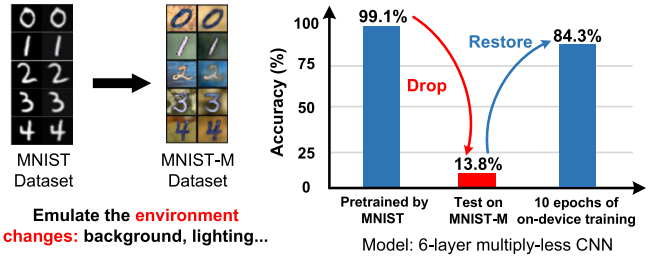


Fig. 23. Inference accuracy recovery by ODFC.

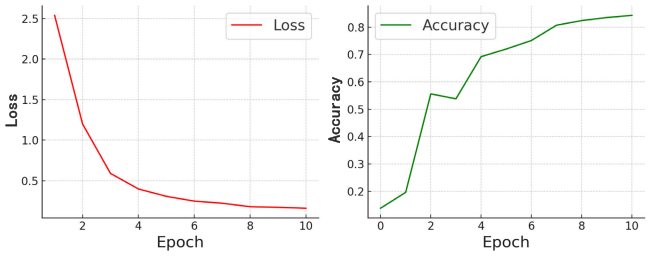


Fig. 24. Training loss and inference accuracy on the MNIST-M dataset.

weights deployed to the same CIM core are trained to be more concentrated within a smaller data range. As a result, the shift range of the weights is smaller than that of the inputs, thereby reducing inference precision loss.

To evaluate the performance of NNs on the proposed chip, a layer-by-layer NN mapping method is adopted. During each layer's computation, as shown in Fig. 22, the 16 columns store the weights' mantissa of 16 output channels, with each column storing the 128 weights' mantissa of different input channels. During inference, 128 inputs with BF16 format are fed into the one-shot CIM engine and computed in parallel with the weights from the 16 columns. Finally, each clock cycle outputs 16 UBF16 format partial sum (Psum) results.

2) NN Performance: Using this model deployment flow, several multiply-less models have been demonstrated on the prototype. In these models, the convolution multiplication is replaced by the $L1$-distance computation. Table II summarizes

TABLE IV
LAYER-WISE ANALYSIS ON THE MNIST-M DATASET

Layer	Weight Size	Latency (ms)	Energy (uJ)*	Energy Eff. (TFLOPS/W)*
Layer-1	3*3*3*32	0.3	0.63	1.37
Layer-2	3*3*32*64	1.71	3.38	1.36
Layer-3	3*3*64*128	2.05	3.42	1.35
Layer-4	3*3*128*256	3.4	3.55	1.30
Layer-5	3*3*256*256	6.8	7.09	1.30
Layer-6	256*10 (FC)	0.04	0.03	1.33
Total	\	14.3	18.1	1.32

*At 0.9V, 180MHz, 25°C. Energy is estimated by post-layout simulation. Off-chip power is not included.

the results of these benchmarks. For the image classification task, the ResNet-20 style multiply-less model for the CIFAR-10 dataset achieves 91.84% inference accuracy, and the ResNet-50 style multiply-less model for the ImageNet dataset achieves 75.00% inference accuracy. It can be seen that there is no accuracy loss compared with the FP32 baseline models. For the object section task, the FCOS style multiply-less model for the COCO dataset achieves 36.8% mAP, with only 0.2% mAP loss compared with the FP32 baseline model. In summary, the NN demonstrations show comparable performance to the FP32 baseline, indicating the effectiveness of algorithm-to-circuit co-optimization.

C. Evaluation for the ODFC

Table III shows the performance of the ODFC. The ODFC occupies an area of 0.037 mm^2 and operates at 180 MHz, matching the peak frequency of the CIM core. Following the training algorithm of the multiply-less NN, $1\times$ is used for gradient computation and weight updating with 16 weights each clock cycle. Thus, the ODFC achieves 16 operations per cycle, resulting in a throughput of 2.88GOPS at BF16 at 180 MHz, 0.9 V.

It is important to note that for ODFC operation, 1 X and 16 gradients are required from off-chip per cycle. To estimate the power consumption, we performed a post-layout simulation, as shown in Table III. The simulated area efficiency and energy efficiency of the ODFC are 0.078 TFLOPS/mm^2 and 1.71 TFLOPS/W at BF16, respectively.

Fig. 23 shows the inference accuracy recovery achieved by ODFC. The inference accuracy drastically drops from 99.1% to 13.8% when the dataset changes. Thanks to the proposed ODFC, after ten epochs of on-device training, the inference accuracy is restored to 84.3%. Although the larger alignment sizes introduce errors to training in pre-aligned-based FP CIM, the accuracy requirements for fine-tuning are much lower than training from scratch, and therefore significant accuracy recovery can still be achieved. Fig. 24 shows the training loss and inference accuracy for each epoch. This shows that on-device model fine-tuning and weight updating by ODFC greatly reduce the accuracy degradation caused by edge environment changes, thereby enhancing its robustness.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:23:27 UTC from IEEE Xplore. Restrictions apply.

TABLE V
COMPARISON TO STATE-OF-THE-ART WORKS

	ISSCC-22 [16]	ISSCC-23 [20]	ISSCC-23 [17]	ISSCC-23 [14]	ESSCIRC-23 [21]	This work	
Technology	28nm	28nm	28nm	28nm	28nm	28nm	
Array Size	96Kb	64Kb	N/A	16Kb	4Kb	16Kb	
NN Operation	Inference	Inference	Inference	Inference	Inference	Inference / On-Device Training	
Cell Type	N/A	6T	N/A	8T (compact)	8T/14T	6T (logic)	
Macro Area (mm^2)	0.941	0.146	0.269	0.028	0.71	0.146	
Voltage (V)	0.6-1.0	0.6-0.9	0.397-0.9	0.54-0.9	0.55-1.2	0.55-0.9	
Frequency (MHz)	50-220	N/A	10-400	20-230	650	20-180	
Data Precision	INT8/16, BF16, FP32	INT8, BF16	INT4/8, FP16, BF16	INT2-8	FP8	INT8	UBF16
Macro Area Efficiency[a] (TOPS/mm^2 or TFLOPS/mm^2)	1.4 (INT8) 1.15 (BF16)	2.05 (BF16)	3.2 (FP16)	4.2 (INT8)	0.94	8.76[c]	7.02[c]
Macro Energy Efficiency[a] (TOPS/W or TFLOPS/W)	57.8 (INT8) 46.2 (BF16)	44 (INT8) 31.6 (BF16)	68.7 (INT8) 12.7 (FP16)	102 (INT8)	12.1	58[c]-152[d]	48[c]-128[d]
FoM[b]	850 (BF16)	1036 (BF16)	881 (FP16)	3427 (INT8)	91 (FP8)	14377 (UBF16)	

[a]Maximum reported value in [1-5] for fair comparison, including FP conversion and CIM macro computation.
[b]FoM = data precision (bit) × area efficiency (TOPS/mm2 or TFLOPS/mm2) × energy efficiency (TOPS/W or TFLOPS/W).
[c]Measured at 0.9V, 180MHz, 25°C, 10% input sparsity and 50% weight sparsity, ODFC core is not included.
[d]Measured at 0.55V, 20MHz, 25°C, 10% input sparsity and 50% weight sparsity, ODFC core is not included.

Table IV presents a layer-wise analysis on the MNIST-M dataset for on-device fine-tuning. Both latency and energy metrics encompass four steps: weight read, weight update, gradient backward, and weight write. The total latency and energy of the six-layer multiply-less NN are 14.3 ms and 18.1 μJ, respectively. Since the complexity of training is significantly higher than inference, the total energy efficiency of training is 1.32 TFLOPS/W at BF16, which is lower than that of inference. In practice applications, on-chip fine-tuning occurs far less frequently compared with inference, leading to a negligible system-level energy efficiency impact.

D. Comparison to State-of-the-Art Works

Table V shows comparison of this work with other state-of-the-art CIM chips. This work achieves the highest area efficiency and energy efficiency compared with other works fabricated with similar technologies. Compared with the bit-parallel multiply-based FP CIM in [22], the one-shot compute scheme proposed in this work significantly reduces the area and energy of the digital logic circuits, thereby improving energy efficiency and area efficiency. Compared with the state-of-the-art FP CIM works [21], the energy efficiency and area efficiency are improved by 4.1× and 3.6×, and the FoM (calculated by data precision × area efficiency × energy efficiency) is improved by 14.7×. Moreover, compared with other FP CIMs works that only support inference, the on-chip fine-tuning core proposed in this work enables online model updates, maintaining high inference accuracy when the edge environment changes.

VII. CONCLUSION

Recent works have extended CIM to high-precision FP operations with additional pre-aligning circuits, enhancing the capability to handle complex tasks on the edge. However, prior FP CIMs still face several major challenges in meeting today's edge intelligence requirements. This work proposes an energy- and area-efficient CIM engine for one-shot pipelined FP NN inference and on-device fine-tuning. To increase throughput and utilization for FP operations, a one-shot compute scheme with a parallel minimum selector (ParMS) is proposed, achieving pipelined FP computation within one cycle and 100% hardware utilization. To decrease the area and energy overhead of FP/INT conversion process, an input–weight co-alignment workflow is proposed to eliminate the exponent selection and simplify the mantissa shifting logic. To reduce the accuracy loss induced by application changes, an ODFC is designed for online model updates. The fabricated 28-nm chip achieves a peak energy efficiency of 128 TFLOPS/W and an area efficiency of 7.44 TFLOPS/mm^2 for UBF16 operation, improving the state-of-the-art by 4.1× and 3.6×, respectively.

REFERENCES

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 2, pp. 84–90, Jun. 2012.
[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
[3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 779–788.
[4] A. Vaswani et al., "Attention is all you need," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.* Red Hook, NY, USA: Curran Associates, 2017, pp. 6000–6010.
[5] J. Zhang, Z. Wang, and N. Verma, "In-memory computation of a machine-learning classifier in a standard 6T SRAM array," *IEEE J. Solid-State Circuits*, vol. 52, no. 4, pp. 915–924, Apr. 2017.
[6] X. Si et al., "A twin-8T SRAM computation-in-memory macro for multiple-bit CNN-based machine learning," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2019, pp. 396–398.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:23:27 UTC from IEEE Xplore. Restrictions apply.

[7] J. Song et al., "A 4-bit calibration-free computing-in-memory macro with 3T1C current-programed dynamic-cascode multi-level-cell eDRAM," *IEEE J. Solid-State Circuits*, vol. 59, no. 3, pp. 842–854, Mar. 2024.

[8] H. Jia et al., "Scalable and programmable neural network inference accelerator based on in-memory computing," *IEEE J. Solid-State Circuits*, vol. 57, no. 1, pp. 198–211, Jan. 2022.

[9] H. Valavi, P. J. Ramadge, E. Nestler, and N. Verma, "A 64-tile 2.4-Mb in-memory-computing CNN accelerator employing charge-domain compute," *IEEE J. Solid-State Circuits*, vol. 54, no. 6, pp. 1789–1799, Jun. 2019.

[10] D. Wang, C. T. Lin, G. K. Chen, P. Knag, R. K. Krishnamurthy, and M. Seok, "DIMC: 2219TOPS/W 2569F2/b digital in-memory computing macro in 28 nm based on approximate arithmetic hardware," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2022, pp. 266–268.

[11] Y.-D. Chih et al., "An 89TOPS/W and 16.3TOPS/mm^2 all-digital SRAM-based full-precision compute-in memory macro in 22 nm for machine-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 252–254.

[12] H. Fujiwara et al., "A 5-nm 254-TOPS/W 221-TOPS/mm^2 fully-digital computing-in-memory macro supporting wide-range dynamic-voltage-frequency scaling and simultaneous MAC and write operations," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2022, pp. 1–3.

[13] H. Mori et al., "A 4 nm 6163-TOPS/W/b 4790-TOPS/mm^2/b SRAM based digital-computing-in-memory macro supporting bit-width flexibility and simultaneous MAC and weight update," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 132–134.

[14] Y. He et al., "A 28 nm 38-to-102-TOPS/W 8b multiply-less approximate digital SRAM compute-in-memory macro for neural-network inference," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 130–132.

[15] B. Yan et al., "A 1.041-Mb/mm^2 27.38-TOPS/W signed-INT8 dynamic-logic-based ADC-less SRAM compute-in-memory macro in 28 nm with reconfigurable bitwise operation for AI and embedded applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 188–190.

[16] H. Diao et al., "A multiply-less approximate SRAM compute-in-memory macro for neural-network inference," *IEEE J. Solid-State Circuits*, early access, Aug. 5, 2024, doi: 10.1109/JSSC.2024.3433417.

[17] F. Tu et al., "A 28 nm 29.2TFLOPS/W BF16 and 36.5TOPS/W INT8 reconfigurable digital CIM processor with unified FP/INT pipeline and bitwise in-memory booth multiplication for cloud deep learning acceleration," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.

[18] J. Yue et al., "A 28 nm 16.9–300TOPS/W computing-in-memory processor supporting floating-point NN inference/training with intensive-CIM sparse-digital architecture," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 1–3.

[19] H. Diao et al., "A 28 nm 128TFLOPS/W computing-in-memory engine supporting one-shot floating-point NN inference and on-device fine-tuning for edge AI," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, Apr. 2024, pp. 1–2.

[20] P.-C. Wu et al., "A 22 nm 832Kb hybrid-domain floating-point SRAM in-memory-compute macro with 16.2–70.2TFLOPS/W for high-accuracy AI-edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 126–128.

[21] A. Guo et al., "A 28 nm 64-kb 31.6-TFLOPS/W digital-domain floating-point-computing-unit and double-bit 6T-SRAM computing-in-memory macro for floating-point CNNs," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 128–130.

[22] J. Saikia, A. Sridharan, I. Yeo, S. Venkataramanaiah, D. Fan, and J.-S. Seo, "FP-IMC: A 28 nm all-digital configurable floating-point in-memory computing macro," in *Proc. IEEE 49th Eur. Solid State Circuits Conf. (ESSCIRC)*, Sep. 2023, pp. 405–408. [Online]. Available: https://api.semanticscholar.org/CorpusID:263707414

[23] C.-T. Lin, J. Oh, K. Lee, and M. Seok, "STAR-SRAM: 43.06-TFLOPS/W, 1.89-TFLOPS/mm^2, 400-Kb/mm^2 floating-point SRAM-based digital computing-in-memory macro in 28-nm CMOS," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, Apr. 2024, pp. 1–2.

[24] B. Zhang, S. Moon, and M. Seok, "A 1-TFLOPS/W, 28-nm deep neural network accelerator featuring online compression and decompression and BF16 digital in-memory-computing hardware," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, Apr. 2024, pp. 1–2.

[25] H. Chen et al., "AdderNet: Do we really need multiplications in deep learning?" in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 1465–1474.

[26] H. Chen, Y. Wang, C. Xu, C. Xu, C. Xu, and T. Zhang, "Universal adder neural networks," 2021, *arXiv:2105.14202*.

[27] H. Shu, J. Wang, H. Chen, L. Li, Y. Yang, and Y. Wang, "Adder attention for vision transformer," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, vol. 34, 2021, pp. 19899–19909.

[28] X. Chen, C. Xu, M. Dong, C. Xu, and Y. Wang, "An empirical study of adder neural networks for object detection," in *Proc. 35th Int. Conf. Neural Inf. Process. Syst.* Red Hook, NY, USA: Curran Associates Inc., 2024, Paper 528.

[29] D. Song, Y. Wang, H. Chen, C. Xu, C. Xu, and D. Tao, "AdderSR: Towards energy efficient image super-resolution," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 15643–15652.

[30] B. Li, Y. Xin, C. Li, Y. Bao, F. Meng, and Y. Liang, "AdderIC: Towards low computation cost image compression," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2022, pp. 2030–2034.

[31] Y. Nie, K. Han, H. Diao, C. Liu, E. Wu, and Y. Wang, "Redistribution of weights and activations for AdderNet quantization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, vol. 35, Jun. 2022, pp. 22739–22751.



Haikang Diao (Graduate Student Member, IEEE) received the B.Eng. degree in electronic engineering and the M.S. degree in circuits and systems from Fudan University, Shanghai, China, in 2019 and 2022, respectively. He is currently pursuing the Ph.D. degree with the School of Integrated Circuits, Peking University, Beijing, China.

His research interests include computing-in-memory, model compression, and acceleration.



Haoyang Luo (Graduate Student Member, IEEE) received the B.S. degree from the School of Information and Electronics, Beijing Institute of Technology, Beijing, China, in 2021.

He is currently working toward the Ph.D. degree with the School of Integrated Circuits, Peking University, Beijing. His research interests include computing-in-memory, computing architecture, and hardware security.



Jiahao Song (Member, IEEE) received the B.Sc. degree (Hons.) from the University of Electronic Science and Technology of China (UESTC), Chengdu, China, in 2017, and the Ph.D. degree from Peking University, Beijing, China, in 2022.

He was a Post-Doctoral Researcher at Peking University, from 2022 to 2024. He is currently working as a Post-Doctoral Researcher with the University of California at San Diego, La Jolla, CA, USA. His research interests include memories, computing in-memory, and low-power mixed-signal circuits design.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:23:27 UTC from IEEE Xplore. Restrictions apply.



Bocheng Xu (Graduate Student Member, IEEE) received the B.Sc. degree from South China University of Technology (SCUT), Guangzhou, Guangdong, China, in 2022. He is currently pursuing the Ph.D. degree with Peking University, Beijing, China.

His current research interests include in-memory computing, ising machine, and combinatorial optimization problems.



Runsheng Wang (Member, IEEE) received the B.S. and Ph.D. degrees (Hons.) from Peking University, Beijing, China, in 2005 and 2010, respectively.

From November 2008 to August 2009, he was a Visiting Scholar at Purdue University, West Lafayette, IN, USA. He joined Peking University, in 2010, where he is currently working as a Full Professor with the School of Integrated Circuits and also working as an Associate Dean of the School of Electronics Engineering and Computer Science. He has authored or co-authored one book, four book chapters, and more than 190 scientific articles, including more than 40 articles published in the International Electron Devices Meeting (IEDM) and Symposium on VLSI Technology (VLSI-T). He has been granted more than 20 U.S. patents and more than 30 Chinese patents. His current research interests include nanoscale CMOS devices, characterization and reliability, design–technology co-optimization (DTCO) and EDA, and emerging technologies and circuits for new-paradigm computing.

Dr. Wang was awarded the IEEE EDS Early Career Award by the IEEE Electron Device Society (EDS), the National Distinguished Young Scholars by the National Natural Science Foundation of China (NSFC), and the Natural Science Award (First Prize) by the Ministry of Education (MOE) of China. He serves on the Editorial Board of IEEE TRANSACTIONS ON ELECTRON DEVICES and *Science China Information Sciences*. He has also served on the Technical Program Committee for many IEEE conferences, including IEDM, IRPS, EDTM, and IPFA.



Yuan Wang (Member, IEEE) received the Ph.D. degree in microelectronics from Peking University, Beijing, China, in 2006.

He is currently working as a Full Professor with the School of Integrated Circuits, Peking University. He has authored or co-authored more than 100 technical articles and conference contributions, including more than 30 papers published in IEEE TRANSACTIONS and journals. He has been granted more than 70 patents. His current research interests include CMOS memory circuits, neuromorphic computing, in-memory computing, and spiking neural networks.



Xiyuan Tang (Senior Member, IEEE) received the B.Sc. degree (Hons.) from the School of Microelectronics, Shanghai Jiao Tong University, Shanghai, China, in 2012, and the M.S. and Ph.D. degrees in electrical engineering from The University of Texas at Austin, Austin, TX, USA, in 2014 and 2019, respectively.

He was a Design Engineer at Silicon Laboratories, Austin, from 2014 to 2017, where he was involved in the RF receiver design. From 2019 to 2021, he was a Post-Doctoral Researcher at The University of Texas at Austin. He is currently working as an Assistant Professor with Peking University, Beijing, China. His research interests include digitally assisted data converters, low-power mixed-signal circuits, and analog data processing.

Dr. Tang serves on the Technical Program Committees (TPC) of ISSCC. He also serves as an Associate Editor for IEEE SOLID-STATE CIRCUITS LETTERS. He was a recipient of IEEE Solid-State Circuits Society Rising Stars in 2020, the Best Paper Award at Silicon Labs Tech Symposium in 2016, the National Scholarship in 2011, and Shanghai Scholarship in 2010.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:23:27 UTC from IEEE Xplore. Restrictions apply.